

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarım ve
Ağ Performans Analiz Platformu

Bursa Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü
Bilgisayar Ağları Dersi — Dönem Projesi

Halil ALPAK
2025-2026

İçindekiler

- 1. Projenin Amacı ve Kapsamı
- 2. Sistem Mimarisi
- 3. Protokol Tasarımı
- 4. Stop-and-Wait Gerçeklemesi
- 5. Go-Back-N (Kayan Pencere) Gerçeklemesi
- 6. Trafik İzleme ve Loglama
- 7. Performans Analizi ve Deney Sonuçları
- 8. Karşılaştırmalı Değerlendirme
- 9. Zorluklar ve Çözümler
- 10. Sonuç ve Gelecek Çalışmalar

Projenin Amacı

Bu projenin amacı, bilgisayar ağıları dersinde ele alınan temel kavramları uygulamalı olarak pekiştirmektir. UDP protokolü üzerinde uygulama katmanında güvenilirlik mekanizmaları tasarlanmış; ağ performansı ölçülmüş ve görselleştirilmiştir.

- UDP ve TCP arasındaki temel farkların uygulamalı incelenmesi
- Sequence number, ACK, timeout ve retransmission mekanizmalarının sıfırdan tasarımı
- Stop-and-Wait ve Go-Back-N protokollerinin karşılaştırılması
- Throughput, goodput, RTT ve retransmission rate metriklerinin ölçülmesi
- Teknik rapor ve sunum hazırlama becerisi kazanımı

Sistem Mimarisi

Sistem istemci-sunucu mimarisinde çalışmakta olup tüm iletişim UDP soket programlama ile gerçekleştirilmektedir. Üç ana katman mevcuttur:

Katman	Bileşen	Görev
Paket Katmanı	packet.py	Serileştirme, MD5 checksum, paket türleri
Aktarım Katmanı	client.py / client_gbn.py	S&W; ve GBN protokol mantığı, kayıp simülasyonu
Alıcı Katman	server.py	Paket kabulü, sıralama, MD5 bütünlük doğrulama
Loglama Katmanı	logger.py	CSV olay kaydı: SEND, RECV, TIMEOUT, FAILED
Analiz Katmanı	analysis.py	Metrik hesaplama, karşılaştırmalı grafik üretimi

Protokol Tasarımı

Paket Türleri:

Tür	Alanlar	Boyut
DATA	Type(1B), SeqNum(4B), TotalPkts(4B), PayloadLen(2B), Checksum(16B) + Payload	27 B + payload
ACK	Type(1B), AckNum(4B), Checksum(16B)	21 B
START	Type(1B), SeqNum=0, TotalPkts(4B), Checksum(16B)	27 B
END	Type(1B), SeqNum=N, Checksum(16B)	27 B

Aktarım Akışı:

- Handshake: İstemci START gönderir → Sunucu ACK(0) yanıtlar
- Veri: Her chunk ayrı DATA paketi olarak gönderilir → Sunucu ACK(seq) yanıtlar
- Kapanış: İstemci END gönderir → Sunucu ACK(N) yanıtlar, dosyayı kaydeder
- Bütünlük: Her pakette MD5 checksum; aktarım sonunda dosyanın tamamı için MD5 karşılaştırması

Stop-and-Wait Gerçeklemesi

Stop-and-Wait protokolünde istemci her paketi gönderir ve ACK gelmeden bir sonraki pakete geçmez. Kayıp veya timeout durumunda paket yeniden gönderilir.

- Chunk boyutu: 1024 bayt
- Varsayılan timeout: 0.5 saniye (yapılandırılabilir)
- Maks. yeniden deneme: **5 deneme/paket** (PDF zorunluluğu)
- 5 denemeden sonra başarısız paket FAILED olarak loglanır ve geçilir
- Aktarım sonunda kayıp paket sayısı ekrana yazdırılır
- İstemci ve sunucu ayrı ayrı MD5 yazdırır — manuel karşılaştırma imkânı

Çalıştırma örneği:

```
python server.py  
python client.py --file donem_projesi.pdf --loss 0.1 --timeout 0.5
```

Go-Back-N Gerçeklemesi

Go-Back-N protokolünde istemci, ACK beklenmeksizin pencere boyutu kadar paketi art arda gönderir. Aynı bir alıcı thread ACK'leri eşzamanlı işler.

- Kayan pencere boyutu: yapılandırılabilir (varsayılan: 5)
- Alıcı thread ayrı çalışır; gönderici thread ile lock ile senkronize edilir
- Kümülatif ACK: ACK(n) alınırsa base n+1'e ilerler, timer yeniden başlar
- Timeout: base pencereden geri gönderim başlatılır
- Maks. yeniden deneme: **5 deneme/paket**; aşılsa base ilerletilir
- Başarısız paket sayısı loglanır ve kullanıcıya raporlanır

Çalıştırma örneği:

```
python client_gbn.py --file donem_projesi.pdf --loss 0.1 --window 5
```

Trafik İzleme ve Loglama

Her aktarım başlatıldığında otomatik olarak CSV log dosyası oluşturulur. Tüm ağ olayları zaman damgasıyla kaydedilir.

Olay	Açıklama
SEND	Paket gönderim zamanı, seq_num, payload_size
RECV	ACK alım zamanı ve ack_num
TIMEOUT	Zaman aşımı oluştuğunda seq_num kaydı
FAILED	5 denemede iletilememiş paketin seq_num kaydı

Log dosyası adlandırma şeması:

```
S&W; : log_loss_<kaynak>_timeout_<timeout>.csv
GBN : log_gbn_loss_<kaynak>_win_<pencere>.csv
```

Performans Metrikleri

Metrik	Formül	Birim
Throughput	$(\text{Toplam gönderilen bit}) / \text{süre}$	bps / kbps
Goodput	$(\text{Benzersiz iletilen bit}) / \text{süre}$	bps / kbps
Retransmission Rate	$(\text{Toplam SEND} - \text{Benzersiz paket}) / \text{Benzersiz paket}$	%
Ortalama RTT	$\text{Ort}(\text{ACK zamanı} - \text{SEND zamanı}), \text{ her paket için}$	ms
Tamamlanma Süresi	$\text{max}(\text{timestamp}) - \text{min}(\text{timestamp})$	s

Deney Senaryoları

Senaryo A — Farklı Kayıp Oranları (S&W;)

- Kayıp: %0, %10, %20, %30
- Beklenti: Kayıp arttıkça throughput/goodput düşer, retransmission rate artar

Senaryo B — S&W; vs Go-Back-N Karşılaştırması

- Aynı kayıp oranında (%10) iki protokol test edilir
- Beklenti: GBN yüksek throughput, S&W; daha basit ama daha yavaş

Senaryo C — Timeout Değerinin Etkisi

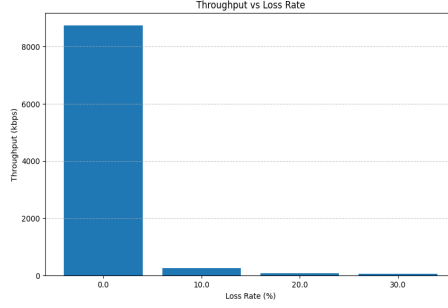
- Timeout: 0.2 s, 0.5 s, 1.0 s
- Beklenti: Çok kısa timeout gereksiz retransmission üretir

Senaryo D — Pencere Boyutunun Etkisi (GBN)

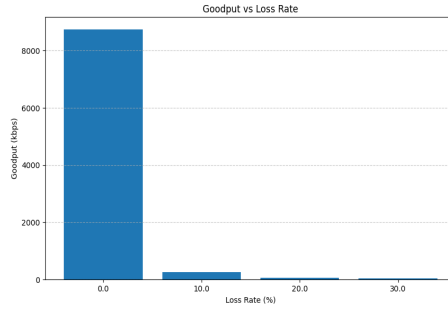
- Pencere: 1, 5, 10
- Beklenti: Büyük pencere throughput'u artırır, hata maliyetini de artırır

Deney Sonuçları — Grafikler

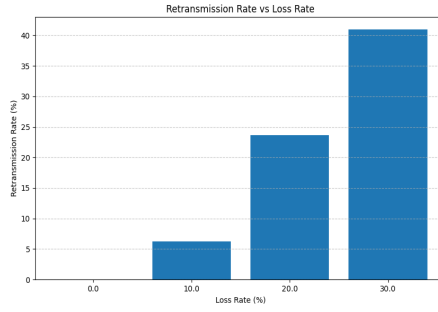
analysis.py çalıştırıldığında log dosyalarından otomatik olarak aşağıdaki grafikler üretilmektedir. Tüm grafikler .png formatında projeye eklenir.



Throughput vs Kayıp Oranı (S&W;)



Goodput vs Kayıp Oranı (S&W;)



Retransmission Rate vs Kayıp Oranı (S&W;)

S&W; vs GBN Karşılaştırmalı Değerlendirme

Kriter	Stop-and-Wait	Go-Back-N
Throughput	Düşük (tek paket bekler)	Yüksek (pencere dolu gönderir)
Kayıp durumu	Sadece kayıp paketi yeniden gönderir	Tüm pencereyi geri gönderir
Karmaşıklık	Basit (tek thread)	Karmaşık (çoklu thread, lock)
Gecikme etkisi	RTT her pakette hissedilir	RTT pencere boyutuyla örtüşür
Düşük kayıp (%0–5)	Kabul edilebilir performans	Belirgin avantaj
Yüksek kayıp (>%20)	Çok yavaş	GBN de ciddi düşüş gösterir

Karşılaşılan Zorluklar ve Çözümler

Problem: GBN'de kümülatif ACK yönetimi

Çözüm: base ve next_seq değişkenleri lock ile korunarak güncellendi

Problem: Thread senkronizasyonu (gönderici-alıcı)

Çözüm: threading.Lock() ile kritik bölgeler izole edildi

Problem: Per-paket maks. yeniden deneme sayısı

Çözüm: retry_counts dict'i ile her paketin deneme sayısı izlendi

Problem: Dosya bütünlüğü doğrulaması

Çözüm: Per-paket MD5 + tüm dosya MD5 karşılaştırması eklendi

Problem: GBN loglarının analiz edilememesi

Çözüm: analysis.py log_gbn_*.csv dosyalarını da tarayacak şekilde güncellendi

Bonus Özellikler ve Gelecek Çalışmalar

Uygulanan Bonus Özellikler:

- ☐ Go-Back-N kayan pencere protokolü gerçekleştirildi
- ☐ Yapay kayıp ve gecikme simülasyon modülü
- ☐ S&W; ve GBN karşılaştırmalı analiz grafikleri
- ☐ RTT ölçümü (SEND-ACK zaman farkı istatistiği)
- ☐ Başarısız paket takibi ve raporlama (FAILED event)

Gelecekte Yapılabilecek Geliştirmeler:

- Selective Repeat — sadece kayıp paketi yeniden gönder
- Congestion control (AIMD) ile TCP benzeri tıkanıklık yönetimi
- Gerçek zamanlı görselleştirme paneli
- Wireshark / pcap tabanlı analiz desteği
- Çoklu istemci desteği

Sonuç

Bu proje kapsamında UDP üzerinde tam işlevsel, güvenilir bir dosya aktarım sistemi başarıyla geliştirilmiştir. Temel zorunlu gereksinimler eksiksiz karşılanmıştır; Go-Back-N ve karşılaştırmalı analiz ise bonus özellik olarak sisteme dahil edilmiştir.

- Stop-and-Wait: basit, güvenilir; yüksek RTT'de verimsiz
- Go-Back-N: yüksek throughput; ancak pencere geri gönderimi ek yük getirir
- Kayıp oranı %10 üzerine çıktığında her iki protokolde ciddi performans düşüşü gözlemlendi
- Timeout süresi çok kısa tutulursa gereksiz retransmission oluştuğu doğrulandı
- MD5 bütünlük doğrulaması ile veri güvenilirliği garanti altına alındı

Teşekkürler / Sorular?